Topic 9A: Source Coding

Efficient Representation of Data
CSE 4405: Data and Telecommunications

Md. Tanvir Hossain Saikat, Jr. Lecturer

Islamic University of Technology (IUT)

Learning Objectives

1. Explain why source coding removes unnecessary redundancy.
2. Distinguish lossless and lossy source coding with communication examples.
3. Compute entropy for a small discrete source and interpret the unit bits/symbol.
4. Explain fixed-length coding, variable-length coding, prefix codes, and instantaneous decoding.
5. Construct a small Huffman code and compute its average code length.
6. Connect source coding to earlier CSE 4405 topics: PCM, DPCM, ADPCM, and multimedia compression.

The Communication-System Picture

- Shannon's model starts with an information source, transmitter, channel, receiver, and destination.
- Source coding sits near the information source: it changes the representation before transmission.
- Channel coding sits near the channel: it adds protection before the noisy part of the system.

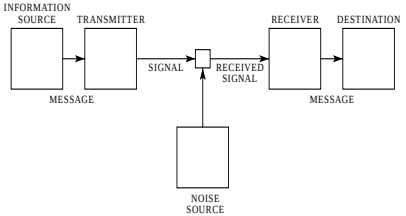
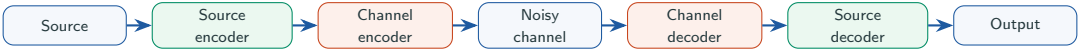


Fig. 1—Schematic diagram of a general communication system.

a decimal digit is about $3\frac{1}{7}$ bits. A digit wheel on a desk computing machine has ten stable positions and

Shannon's original communication-system figure, cropped from the 1948 paper.

Where Source Coding Sits



Source coding

Use fewer bits for the same information, or for an acceptable approximation.

Channel coding

Use extra controlled bits so the receiver can detect or correct channel errors.

What Redundancy Means Here

- Raw data often repeats patterns or carries predictable structure.
- Statistical structure in the original message is exactly what makes savings possible.
- For English-like sources, letters and letter pairs are not equally likely: think of common versus uncommon letters and digrams.
- Source coding exploits that predictability so the channel does not carry avoidable bits.

Operational reading

If the receiver can infer a pattern from fewer bits plus an agreed codebook, the omitted bits were not carrying new surprise.

Lossless vs. Lossy Source Coding

Type	Receiver requirement	Typical examples
Lossless	Reconstruct the original data exactly	ZIP, PNG, text compression
Lossy	Reconstruct an acceptable approximation	JPEG, MP3, video compression

- This course emphasizes lossless source coding first because it connects cleanly to bits, frames, and data communication.
- Lossy coding is still important for audio, images, and video, but full perceptual coding belongs beyond this module.

Entropy: The Quantity Source Coding Cares About

Definition for a discrete memoryless source

If source X emits symbols x_i with probabilities p_i, its entropy is

$$H(X) = -\sum_i p_i \log_2 p_i.$$

The unit is bits per source symbol.

- The logarithm base selects the unit; base 2 gives bits.
- Entropy is a measure of information, choice, and uncertainty.
- This formula is the first mathematical tool used throughout the rest of the module.

Entropy Is an Average, Not a Per-Symbol Length

$$H(X) = \mathbb{E}\big[-\log_2 P(X)\big]$$

- A rare symbol has larger self-information $-\log_2 p$.
- A common symbol has smaller self-information.
- Entropy averages those values using the source probabilities.

Two-symbol intuition

A fair coin has one bit of uncertainty per flip. A very biased coin has less than one bit of uncertainty per flip because the next outcome is easier to guess.

Worked Example: Binary Entropy

Given: $P(0) = 0.9$, $P(1) = 0.1$.

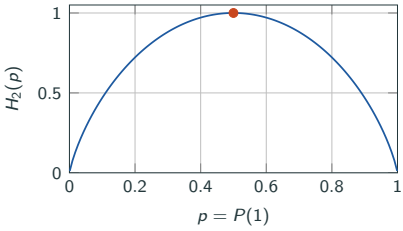
$$\begin{aligned} H(X) &= -\left(0.9 \log_2 0.9 + 0.1 \log_2 0.1\right) \\ &= -\left(0.9(-0.1520) + 0.1(-3.3219)\right) \\ &= 0.1368 + 0.3322 \\ &\approx 0.469 \text{ bits/symbol.} \end{aligned}$$

Interpretation

The physical stream may store each symbol in one bit, but the average uncertainty is about 0.469 bits/symbol. That gap is the reason compression is possible in principle.

When Is Entropy High or Low?

- Entropy is zero only when one outcome is certain.
- For a fixed alphabet size n, entropy is maximum at equal probabilities and equals $\log_2 n$.
- Moving probabilities toward equality increases entropy.



Binary entropy $H_2(p)$, maximum at $p = 0.5$.

Fixed-Length Coding

Symbol	Fixed code
A	00
B	01
C	10
D	11

- Four symbols require $\lceil \log_2 4 \rceil = 2$ bits/symbol if every codeword has the same length.
- Fixed-length coding is simple because the decoder cuts the bitstream into equal chunks.
- It ignores probability: A gets two bits even if A is very common.

Variable-Length Coding

Core move

Give short codewords to frequent symbols and longer codewords to rare symbols. The average length can drop even though some individual codewords become longer.

Symbol	Probability	Candidate code
A	0.40	0
B	0.30	10
C	0.20	110
D	0.10	111

Prefix Codes: Why They Matter

Prefix code

No valid codeword is the beginning of another valid codeword. This makes symbol boundaries discoverable during decoding.

Valid prefix code

$$A = 0, \quad B = 10, \quad C = 110, \quad D = 111$$

The decoder can read left to right and stop as soon as a valid leaf is reached.

Non-prefix problem

$$A = 0, \quad B = 01, \quad C = 011$$

After seeing 0, the receiver cannot know whether the symbol has ended.

Decoding a Prefix Code

Codebook: $A = 0,\ B = 10,\ C = 110,\ D = 111.$

Bitstream: $0\,10\,110\,111\,0$

Decoded symbols: $A\,B\,C\,D\,A$

- The receiver does not need commas in the actual transmitted bitstream.
- Prefix structure makes the commas recoverable from the code itself.
- This is why a variable-length source code must be designed, not just guessed.

Expected Code Length

Average length

For codeword lengths ℓ_i and symbol probabilities p_i,

$$L_{\text{avg}} = \sum_i p_i \ell_i.$$

For the candidate code

$$L_{\text{avg}} = 0.4(1) + 0.3(2) + 0.2(3) + 0.1(3) = 1.9 \text{ bits/symbol}.$$

What improved

The fixed-length code used 2 bits/symbol. The variable-length code saves 0.1 bit per symbol on average for this probability model.

Entropy as a Lower-Bound Intuition

- Shannon's noiseless coding theorem connects source entropy to the channel capacity required by efficient coding.
- For this undergraduate module, use the operational idea: good lossless source codes try to push average length close to entropy.
- Huffman coding is optimal among symbol-by-symbol prefix codes for a finite ensemble.

A practical target is $H(X) \leq L_{\mathrm{avg}}$, with L_{avg} close to $H(X)$ when the code matches the probabilities well.

Huffman Coding: What Problem It Solves

Huffman coding

Huffman coding constructs a minimum-redundancy prefix code for a finite set of messages with known probabilities.

- The method minimizes the average number of coding digits per message for a finite ensemble.
- The code remains prefix-decodable because the construction creates a tree with symbols at leaves.
- More probable symbols are never assigned longer codewords than less probable symbols in an optimum code.

Original Huffman Construction Image

- The original paper shows repeated combination of low-probability messages.
- Each combination reduces the active ensemble by one for binary coding.
- Codes are read by assigning branch digits after the tree is formed.

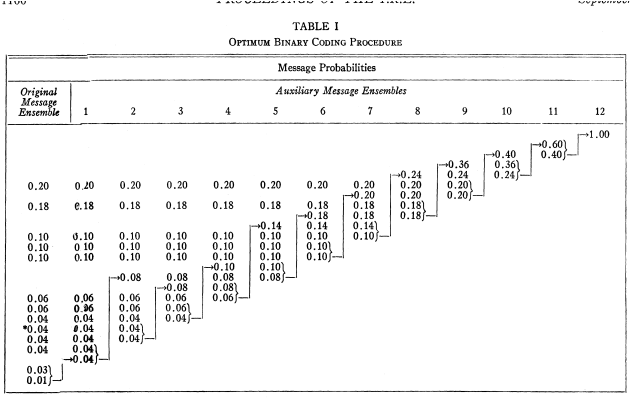


TABLE I

OPTIMUM BINARY CODING PROCEDURE

Original Message Ensemble	Message Probabilities: Auxiliary Message Ensembles											
	1	2	3	4	5	6	7	8	9	10	11	12
0.20	0.20	0.20	0.20	0.20	0.20	0.20	0.20	0.24	0.36	0.40	0.60	1.00
0.18	0.18	0.18	0.18	0.18	0.18	0.18	0.20	0.20	0.24	0.36	0.40	
0.10	0.10	0.10	0.10	0.10	0.14	0.14	0.18	0.18	0.20	0.24		
0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.14	0.14	0.18			
0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10	0.10				
0.06	0.06	0.06	0.08	0.08	0.10	0.08						
0.06	0.06	0.06	0.06	0.08	0.08							
0.04	0.04	0.04	0.06	0.06								
0.04	0.04	0.04	0.04									
0.04	0.04	0.04										
0.03	0.04											
0.01												

Huffman's Table I, cropped from the original paper.

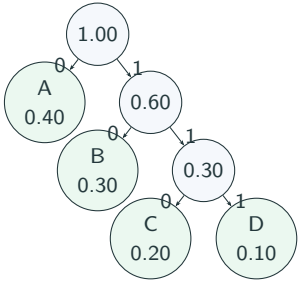
Huffman Algorithm, Step by Step

1. List all symbols with their probabilities.
2. Combine the two least probable symbols into one composite node.
3. Replace them by the composite probability, then sort again if needed.
4. Repeat until a single root remains.
5. Label each pair of branches with 0 and 1.
6. Read each codeword from root to leaf.

Huffman Example: Build the Tree

Symbol	Probability
A	0.40
B	0.30
C	0.20
D	0.10

1. $D(0.10) + C(0.20) \rightarrow 0.30$.
2. $B(0.30) + CD(0.30) \rightarrow 0.60$.
3. $A(0.40) + BCD(0.60) \rightarrow 1.00$.



Tree built by the Huffman rule; branch labels may be swapped.

Huffman Example: Read the Code

Symbol	Probability	Code	Length
A	0.40	0	1
B	0.30	10	2
C	0.20	110	3
D	0.10	111	3

$$L_{\text{avg}} = 0.4(1) + 0.3(2) + 0.2(3) + 0.1(3) = 1.9.$$

$$H(X) = -(0.4 \log_2 0.4 + 0.3 \log_2 0.3 + 0.2 \log_2 0.2 + 0.1 \log_2 0.1) \approx 1.846.$$

How Close Was the Code to Entropy?

$$\eta = \frac{H(X)}{L_{\text{avg}}} = \frac{1.846}{1.900} \approx 0.972.$$

- η is the coding efficiency for this example under this source model.
- The gap $L_{\text{avg}} - H(X) \approx 0.054$ bits/symbol is redundancy left by the code.
- A short, symbol-by-symbol prefix code cannot always hit entropy exactly.

Do not memorize the tree

The important reasoning is: combine rare symbols, put common symbols near the root, then verify the average length.

If the Probabilities Change

Codebooks are matched to models

A Huffman code is optimal for the probabilities used to build it. If the real source distribution changes, the same code may no longer be optimal.

- A source is modelled by symbol probabilities or transition probabilities.
- Huffman's construction starts from ordered message probabilities.
- Therefore the compression story is always tied to a source model, not just an alphabet.

Practical Source-Coding Examples

Application	Source-coding idea
Text compression	Exploit unequal symbol/string frequencies; examples include Huffman, arithmetic coding, and Lempel–Ziv families.
Image compression	Remove spatial redundancy among nearby pixels; lossy image coding may also remove less visible detail.
Audio compression	Represent perceptually important components more carefully than less important components.
Video compression	Use similarity across frames instead of sending each frame independently.
Digital telephony	PCM, DPCM, and ADPCM represent sampled speech more efficiently or robustly.

Connection to Earlier CSE 4405 Topics

- Topic 4 already introduced PCM as sampling, quantization, and binary encoding.
- DPCM and ADPCM reduce the number of bits by coding prediction error or adapting the step behavior instead of always coding absolute samples.
- Source coding gives the general language for why those ideas can reduce bit rate: they exploit structure in the source.
- The output is still a bitstream, so it must later be line-coded, modulated, multiplexed, framed, and protected.

Course relevance

Compression is not a separate software trick here; it changes the offered bit rate that every downstream communication layer must carry.

What Students Should Not Memorize

Do not memorize

- one fixed Huffman tree for every problem;
- a code table without checking prefix property;
- entropy as just another formula.

Reason through

- what is predictable in the source;
- whether exact recovery is required;
- how probabilities determine average length;
- why the receiver can decode unambiguously.

Summary

- Source coding answers: what is the minimum number of bits needed to represent the information?
- Lossless coding requires exact reconstruction; lossy coding allows an acceptable approximation.
- Entropy measures average uncertainty in bits/symbol.
- Fixed-length codes are simple but ignore probability.
- Prefix codes allow instantaneous decoding of variable-length codewords.
- Huffman coding assigns shorter codewords to more probable symbols and minimizes average length among finite symbol-by-symbol prefix codes.

Appendix: First-Exposure Terminology I

Term	Meaning for this module
Source	The entity or process that produces information to be communicated.
Source coding	Encoding the source output so it uses fewer bits for the required reconstruction quality.
Lossless coding	Source coding where the decoder must reproduce the original data exactly.
Lossy coding	Source coding where the decoder may reproduce an approximation.
Discrete source	A source whose output can be modeled as symbols from a finite or countable alphabet.
Probability model	The probabilities assigned to source symbols or symbol sequences.
Entropy	Average uncertainty/information per source symbol, measured in bits/symbol with base-2 logarithms.
Self-information	The quantity $-\log_2 p$, larger for rarer outcomes.

Appendix: First-Exposure Terminology II

Term	Meaning for this module
Codeword	The bit string assigned to one source symbol or message.
Codebook	The agreed mapping from symbols/messages to codewords.
Fixed-length code	A code where every codeword has the same number of bits.
Variable-length code	A code where different symbols/messages may have different codeword lengths.
Prefix code	A variable-length code where no codeword is the prefix of another codeword.
Instantaneous decoding	Decoding as soon as a complete codeword is recognized, without waiting for future bits.
Average code length	Expected number of coded bits per source symbol, $L_{\mathrm{avg}} = \sum_i p_i \ell_i$.
Huffman coding	A minimum-redundancy prefix-code construction for a finite probability model.

Appendix: References

All definitions, formulas, examples, and reproduced figures in this deck are drawn from the following sources.

- Local module: *Short Module: Source Coding, Channel Coding, and Joint Source–Channel Coding*, `Source and Channel Coding.pdf`, 13 pp.
- C. E. Shannon, "A Mathematical Theory of Communication," *Bell System Technical Journal*, vol. 27, pp. 379–423 and 623–656, 1948. Local file: `references/shannon-1948-mathematical-theory.pdf`.
- D. A. Huffman, "A Method for the Construction of Minimum-Redundancy Codes," *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, 1952. Local file: `references/huffman-1952-minimum-redundancy.pdf`.
- B. A. Forouzan, *Data Communications and Networking*, 5th ed., McGraw-Hill, 2013; used here only through already prepared course references for PCM/DPCM/ADPCM and later error-control figures.

Questions?